

[16] Scalable Similarity Search for Big Data

저자: Pavel Zezula (Masaryk University, Czech Republic)

학회/년도: Springer INFOSCALE 2015 (초청 강연)

분량: 초청 논문/서베이, 약 12 페이지

역할군: (D) 이론적 기반

요약

이 서베이 논문은 대규모 유사도 검색의 **이론적 기초**를 제공한다. **메트릭 공간(metric space) 이론**이라는 수학적 근거 위에서 유사도 검색을 정의하며, "왜 벡터 검색의 결과 건수를 예측하기 어려운가"를 이론적으로 설명한다.

핵심 기여:

1. **메트릭 공간의 기하학적 성질**: 고차원 공간에서의 거리 분포 특성 분석
2. **M-tree 인덱스 구조**: 메트릭 공간의 분할 기반 동적 인덱스
3. **고차원의 저주와 카디널리티**: 거리 집중(concentration) 현상의 수학적 분석
4. **분산 환경으로의 확장**: 빅데이터 환경에서의 유사도 검색

이 논문은 Exqutor가 통계 기반이 아닌 실제 인덱스 탐색 기반의 카디널리티 추정을 선택한 이유의 이론적 근거를 제공한다.

상세분석

16.1 해결하고자 하는 문제: 빅데이터 시대의 유사도 검색

세 가지 핵심 도전 과제:

1. 확장성(Scalability):

- 데이터가 수십억 건에서 수조 건으로 증가해도 응답 시간이 허용 범위 내여야 한다.
- 기존 풀 스캔(brute-force) 방식은 $O(n)$ 복잡도이므로, 데이터 크기 n 이 1000 배 증가하면 응답 시간도 1000 배 증가한다. 이는 참을 수 없다.
- 인덱스를 통해 $O(\log n)$ 또는 $O(\log^k n)$ 수준으로 개선해야 한다.

2. 프라이버시 보존(Privacy Preservation):

- 클라우드 아웃소싱 환경에서 원본 데이터는 공개하지 않으면서 유사도 검색을 수행해야 한다.
- 암호화된 데이터에 대한 검색(searchable encryption) 기술이 필요하다.

3. 멀티모달 통합(Multimodal Integration):

- 텍스트, 이미지, 오디오, 비디오 등 서로 다른 모달리티의 데이터를 하나의 통합 프레임워크에서 검색해야 한다.
- 각 모달리티를 공통 벡터 공간으로 매핑하는 임베딩 기술이 필요하다.

16.2 메트릭 공간(Metric Space) 이론: 수학적 기초

메트릭 공간의 정의:

메트릭 공간 (X, d) 는 집합 X 와 거리 함수 $d: X \times X \rightarrow \mathbb{R}$ 로 구성되며, 거리 함수가 다음 **공리**를 만족한다:

1. 양의 정부호성 (Positivity):

- $d(x, y) \geq 0$ (거리는 항상 0 이상)
- $d(x, y) = 0 \iff x = y$ (같은 점 사이의 거리만 0)

1. 대칭성 (Symmetry):

- $d(x, y) = d(y, x)$ (x 에서 y 까지의 거리 = y 에서 x 까지의 거리)

1. 삼각 부등식 (Triangle Inequality):

- $d(x, z) \leq d(x, y) + d(y, z)$ (우회하는 것보다 직접 가는 것이 가깝다)

삼각 부등식의 힘:

삼각 부등식이 핵심이다. 이 성질 덕분에:

$$d(x, z) \leq d(x, y) + d(y, z)$$

만약 $d(x, y) + d(y, z) <$ 현재까지의 최소 거리 라면, $d(x, z)$ 는 이미 본 것보다 작을 수 없다. 따라서 **z 를 검사할 필요 없다.**

이런 식으로 많은 거리 계산을 건너뛰(prune) 수 있어, 전수 검색 없이도 유사도 검색이 가능하다.

예시: 뉴욕 지도에서의 거리

$$\text{삼각 부등식: } d(A, C) \leq d(A, B) + d(B, C)$$

뜻: A에서 C까지의 직선거리 $\leq$ A $\rightarrow$ B $\rightarrow$ C의 경로거리

이 성질을 활용하면:

- A에서 거리 5km 이내인 장소를 찾을 때
- B를 거쳐가는 모든 경로를 검사하는 것이 아니라
- A에서 거리 5km 이내의 "동네" 영역만 검색하면 된다.

16.3 M-tree: 메트릭 공간의 동적 인덱스 구조

M-tree 의 개념:

M-tree 는 B-tree 처럼 균형 잡힌 트리 구조를 메트릭 공간에 적용한 것이다.

```

[루트 - 전체 데이터의 중심점]
/   |   \
[서브트리 1] [서브트리 2] [서브트리 3]
(중심: p1) (중심: p2) (중심: p3)
반경: r1  반경: r2  반경: r3
/ \   / \   / \
[리프]... [리프]... [리프]...

```

각 노드는:

- **중심점(pivot/representative):** 그 부분트리를 대표하는 점
- **반경(covering radius):** 중심점으로부터 이 부분트리의 가장 먼 점까지의 거리

검색 과정:

쿼리 점 q 에서 거리 r 이내의 모든 점을 찾을 때:

1. 루트 노드부터 시작
2. 각 자식 노드에 대해:
 - 삼각 부등식으로 가지 제거(pruning):
 $d(q, \text{자식중심}) - \text{자식반경} > r$ 이면
 $\rightarrow$ 이 가지는 검색할 필요 없음
 - 그 외는 재귀적으로 탐색
3. 리프 노드에서 정확한 거리 계산 및 필터링

M-tree 의 장점:

- 동적 삽입/삭제 지원 (B-tree 처럼)
- 다양한 메트릭(유클리드, 맨해튼, 편집 거리 등) 지원
- 메트릭 공간의 특성만 만족하면 모든 데이터 타입 지원

16.4 고차원의 저주와 거리 분포의 집중(Concentration)

고차원 공간의 기하학적 성질:

차원이 높아질수록 흥미로운(그리고 직관에 반하는) 현상들이 일어난다:

1. 거리의 집중(Distance Concentration):

고차원 공간에서 모든 점 사이의 거리가 거의 같아진다.

예시: 무작위 단위 벡터 1,000 개 생성

차원	최소 거리	최대 거리	차이
2D	0.1	1.9	1.8
10D	1.2	1.5	0.3
100D	1.41	1.42	0.01
1000D	1.414	1.415	0.001

1000 차원에서는 모든 점이 서로 거의 같은 거리(약 $\sqrt{2} \approx 1.414$)에 있다!

수학적 설명:

d 차원 공간에서 무작위 단위 벡터들 사이의 거리 E[D]는:

$$E[D] \approx \sqrt{(2d/\pi) \times (1 - 1/(4d) + O(1/d^2))}$$

$$d \rightarrow \infty \text{ 일 때, 표준편차 } \sigma[D] \approx \sqrt{1/(2d)}$$

거리의 변동 계수(coefficient of variation):

$$CV = \sigma[D] / E[D] \approx \sqrt{\pi/(2d)}$$

이것은 d 에 반비례하므로, 차원이 높아질수록 거리 분포가 더 집중된다.

2. "저주"의 의미:

거리가 집중되면:

모든 데이터 점 사이의 거리가 비슷함

↓

거리 임계값 $d(q, x) < r$ 을 만족하는 점의 비율이 급격히 변함

↓

카디널리티를 정확히 예측하기 극도로 어려움

예: 쿼리 반경 r 을 0.1 증가시키면:

- 저차원: 결과 점 10% 증가
- 고차원: 결과 점 200% 증가 (넓이 vs. 부피 증가의 차이)

16.5 고차원 카디널리티 추정의 어려움

히스토그램 기반 통계의 한계:

관계형 DB 는 스칼라 값의 카디널리티 추정을 위해 **히스토그램**을 사용한다:

값 범위	튜플 개수
-----	-----
0~100	1,000
100~200	2,000
200~300	800

가격이 150 인 상품 수를 추정하려면, 100~200 범위의 히스토그램을 보면 된다.

하지만 **벡터 거리**에는 이런 방식이 작동하지 않는다:

```
WHERE embedding <=> query < 0.1
```

이 조건을 만족하는 벡터의 개수는:

- 데이터 분포에 완전히 의존
- 쿼리 벡터의 위치에 의존
- 임계값의 작은 변화에 극도로 민감

예:

- $r = 0.09$: 결과 1,000 개
- $r = 0.10$: 결과 10,000 개
- $r = 0.11$: 결과 50,000 개

히스토그램을 구성하려면 쿼리 벡터마다 미리 거리를 계산하고 저장해야 하는데, 이는 불가능하다 (쿼리마다 다르기 때문).

16.6 본 논문과의 관계

이 논문이 보여주는 것:

1. 벡터 카디널리티 추정의 이론적 어려움:

고차원 공간에서 거리 분포의 집중 현상이 일어나므로, 기존의 통계 기반 추정(히스토그램)은 근본적으로 작동할 수 없다.

2. Exqutor 가 통계 기반 대신 실제 탐색 기반을 선택한 이유:

카디널리티를 정확히 얻는 유일한 방법은:

- 실제 인덱스를 탐색해서 결과를 세는 것

Exqutor 의 ECQO 는 이 철학을 따른다:

쿼리 플래너 단계 → ECQO 혹 → 실제 인덱스 범위 탐색 (1~2ms)
→ 정확한 카디널리티 얻음 → 옵티마이저에 전달 → 최적 계획

3. 왜 Exqutor 의 1~2ms 오버헤드가 정당한가:

쿼리 실행 시간이 수백 ms ~ 수초인데, 플래너 단계에서 1~2ms 를 투자하여 최적 계획을 얻는 것은 매우 합리적이다.

특히 선택도가 극도로 불확실한 경우(0.1%에서 90%까지 변할 수 있는 경우), 정확한 추정 비용 1~2ms 는 하찮다.

16.7 메트릭 공간 이론과 현대 벡터 검색의 연결

2015 년 논문의 한계:

이 서베이는 2015 년 기준이므로:

- HNSW, DiskANN 같은 최근 그래프 기반 인덱스는 다루지 않음
- 학습된 인덱스(learned index) 개념은 없음
- 신경망 기반 임베딩의 고차원 특성은 다루지 않음

하지만 기본 원리는 여전히 유효:

메트릭 공간의 거리 집중 현상은 변하지 않으므로, HNSW 나 DiskANN 을 사용하더라도 카디널리티 추정 문제는 여전히 존재한다.

추가 제기 문제

1. **적응적 인덱스 구조:** M-tree 는 정적이지만, 데이터가 자주 업데이트되는 환경에서는 재구축 비용이 크다. 동적 데이터 환경에 최적화된 인덱스 구조 연구가 필요하다.
2. **고차원 특화 인덱스:** 메트릭 공간 이론은 일반적이지만, 특정 차원 범위(예: 768 차원 BERT 벡터)에 최적화된 인덱스를 설계하는 것이 가능할까?
3. **카디널리티 상한/하한 추정:** 정확한 값을 얻을 수 없다면, 최소한 상한과 하한을 빠르게 추정할 수 있는 방법이 있을까? 이를 통해 옵티마이저가 비효율적인 선택을 피할 수 있을까?
4. **멀티모달 통합의 현실화:** 논문은 멀티모달 통합을 언급하지만, 실제로 텍스트, 이미지, 오디오를 하나의 메트릭 공간으로 통합하는 것의 실무적 난제는 무엇인가?